City Data Scrapping With Python Regular Expressions

Ονοματεπώνυμο Μαθητη: Μάριος-Βασίλειος Μανθέλας

Αριθμός Μητρώου (Α.Μ.): ics24169

Email: ics24169@uom.edu.gr

Υπεύθυνος Καθηγητής: Γιάννης Ρεφανίδης

Github Repository

Περιεχόμενα

1. Εισαγωγή 2

2. Μεθοδολογία και Εργαλεία Υλοποίησης 2

3. Περιγραφή Κώδικα 2

3.1. Κλάση scraperPage 2

3.2. Κλάση Find 3

4. Πειραματική Διαδικασία και Αξιολόγηση Αποτελεσμάτων 5

4.1. Φάση Ανάπτυξης 5

4.2. Φάση Ελέγχου 6

4.3. Φάση Αξιολόγησης 7

Στιγμιότυπα Εκτέλεσης

Στιγμιοτυπο Εκτελεσης 1 Αθηνα 5

Στιγμιοτυπο Εκτελεσης 2 Θεσσαλονικη 6

Στιγμιοτυπο Εκτελεσης 3 Νεα Ιωνια 6

Στιγμιοτυπο Εκτελεσης 4 Λαρισα 6

Στιγμιοτυπο Εκτελεσης 5 Τρικαλα 6

Στιγμιοτυπο Εκτελεσης 6 Καρδιτσα 7

Στιγμιοτυπο Εκτελεσης 7 Πατρα 7

Στιγμιοτυπο Εκτελεσης 8 Ιωαννινα 7

Στιγμιοτυπο Εκτελεσης 9 Χανια 7

1. Εισαγωγή

Σε αυτήν την εργασία θα ασχοληθούμε με την δημιουργία ενός μικρού προγράμματος σε γλώσσα Python 3.x, με κεντρικό στόχο την ανάλυση και εξαγωγή πληροφοριών από ιστοσελίδες της Wikipedia που αφορούν διάφορες πόλεις του κόσμου. Για την υλοποίηση θα χρησιμοποιήσουμε τις παρακάτω βιβλιοθήκες της Python, Requests , Beautiful Soup και Re modules για την εξαγωγή, την μορφοποίηση και την ανάλυση αντίστοιχα.

2. Μεθοδολογία και Εργαλεία Υλοποίησης

Για να πετύχουμε τον στόχο μας ακολουθήσαμε τα παρακάτω βήματα επεξεργασίας δεδομένων με την χρήση των παρακάτω βιβλιοθηκών:

- **Requests:** Χρησιμοποιήθηκε για την σύνδεση με τους εξυπηρετητές της Wikipedia για την λήψη του ακατέργαστου περιεχομένου της ιστοσελίδας που δόθηκε σαν είσοδος στο πρόγραμμα σε μορφή HTML.
- **Beautiful Soup:** Με τη χρήση της βιβλιοθήκης αυτής αφαιρούμε το ¨άχρηστο¨ κείμενο από τα HTML δεδομένα μας.
- **Re Module:** Τέλος χρησιμοποιήσαμε την βιβλιοθήκη re για την εξαγωγή δεδομένων που τηρούν συγκεκριμένα patterns.

3. Περιγραφή Κώδικα

Στην εργασία αυτήν δημιουργήσαμε και υλοποιήσαμε δύο βασικές κλάσεις, την κλάση scraperPage() και την Find() τις οποίες περιγράφουμε πιο αναλυτικά παρακάτω.

3.1. Κλάση scraperPage

Η κλάση scraperPage δέχεται ως όρισμα ένα αλφαριθμητικό το οποίο αναπαριστά το url της σελίδας μιας πόλης από την Wikipedia. Η κλάση υλοποιεί την μέθοδο getPage() η οποία χρησιμοποιώντας το requests module αποστέλει ένα αίτημα HTTP, με ενσωματωμένα τα σωστά headers, λαμβάνοντας τον πηγαίο κώδικα της σελίδας που αντιστοιχεί στο url που εισάγαμε, καθώς με την βοήθεια του beautiful soup πραγματοποιεί ανάλυση του κώδικα επιστρέφοντας αποκλειστικά το περιεχόμενο κείμενου της σελίδας χωρίς τα html tags, διευκολύνοντας έτσι την περαιτέρω επεξεργασία.

```
class scraperPage:
    def __init__(self, url):
        self.url=url
        self.soup=None

    def getPage(self):
        headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36'
        }

        try:
            response = requests.get(self.url,headers=headers)
            response.raise_for_status()  # Check if the request was successful
            self.soup = BeautifulSoup(response.text, 'html.parser').get_text()
            return self.soup
        except requests.exceptions.RequestException as e:
            print(f"An error occurred: {e}")
```

3.2. Κλάση Find

Η κλάση Find δέχεται ως όρισμα το περιεχόμενο κείμενο της ιστοσελίδας που παραρίχθηκε από την χρήση της scraperPage->getPage. Η κλάση αυτήν υλοποιεί μία μέθοδο για την κάθε πληροφόρία που θέλουμε να εξάγουμε, πράγμα που πετυχαίνει με την χρήση του module RE(κανονικών εκφράσεων), το οποίο εφαρμόζει συγκεκριμένα πρότυπα αναζήτησης πάνω στο κείμενο.

```
class Find:
    def __init__(self, soup):

        self.soup=self.text = soup
        #self.text = self.soup.get_text()

    def extractTokens(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return []
        # Use regex to find words (tokens)
        tokens = re.findall(r'\b[A-Za-z][a-z]+\b', self.text)

        return tokens

    def findName(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return None
        return re.search(r'([A-Z][a-z]+(?: [A-Z][a-z]+)*)', self.text).group(1)
        #return re.search(r'[A-Z][a-z]+',self.text).group()

    def findPopulation(self):
        if self.soup is None:
```

```
            print("Soup object is not initialized.")
            return None
        return re.search(r'Population.*?Municipality\s*(?:\[[0-9]+\])*\s*([0-
9]{1,3}(?:,[0-9]{3})+)', self.text, re.S).group(1)
        #return re.findall(r'[Mm]unicipality([0-9]+,?[0-9]+)',self.text)[1]

    def findArea(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return None
        return re.findall(r'[Mm]unicipality([0-9]+\.?[0-9]+)',self.text)[0]

    def findRegion(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return None
        return re.search(r'(?:Administrative region|Periphery|Region)\s*([A-Z][a-z]+(?:
[A-Z][a-z]+)*)', self.text).group(1)
        #return re.findall(r'Administrative region([A-Z][a-z]+(?: [A-Z][a-
z]+)?)',self.text)[0]

    def findCoordinates(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return None
        return re.findall(r'Coordinates:\s([0-9]+°[0-9]+..*[0-9]+°[0-
9]+.*[NSWE])',self.text)[0]

    def findElevation(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return None
        return re.findall(r'[Ee]levation([0-9]+)',self.text)[0]

    def findTimeZone(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return None
        return re.findall(r'UTC\+[0-9+]',self.text)

    def findTemp(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return None
        tempArray=re.findall(r'-?[0-9]+.?[0-9]\(-?[0-9]+\.?[0-9]\)',self.text)
        return tempArray[12],tempArray[64],tempArray[38]

    def findCountry(self):
        if self.soup is None:
            print("Soup object is not initialized.")
            return None
        return re.findall(r'Country.?([A-Z][a-z]+(?: [A-Z][a-z]+)*)', self.text)[0]
        #return re.findall(r'Country.?([A-Z][a-z]+)',self.text)[0]
```

4. Πειραματική Διαδικασία και Αξιολόγηση Αποτελεσμάτων

4.1. Φάση Ανάπτυξης

Για την φάση της ανάπτυξης των κανονικών εκφράσεων επιλέχθηκαν τρεις πόλεις. Οι πόλεις αυτές επιλέχθηκαν λόγο της διαφορετικής δομής του περιεχομένου τους ώστε να δημιουργήσουμε ευέλικτες κανονικές εκφράσεις που θα αντέχουν σε μεταβολές και ¨θόρυβο¨ δεδομένων.

Οι πόλεις που επιλέχθηκαν είναι:

1. Αθήνα

Η Αθήνα αποτέλεσε τον κεντρικό πυλώνα για τον σχεδιασμό και τη ρύθμιση του συνόλου των παραμέτρων της εφαρμογής. Λόγω της δομής των πληροφοριών της, χρησιμοποιήθηκε ως το βασικό μοντέλο για τη δημιουργία κανονικών εκφράσεων που καλύπτουν κάθε πεδίο: από την εξαγωγή του ονόματος και της χώρας, μέχρι τον εντοπισμό των γεωγραφικών συντεταγμένων και των δεδομένων θερμοκρασίας. Η λογική που αναπτύχθηκε εδώ, όπως η ικανότητα διαχείρισης σύνθετων αριθμητικών δεδομένων με διαχωριστικά χιλιάδων ή ο εντοπισμός συγκεκριμένων ετικετών στο Infobox, καθόρισε τις προδιαγραφές για την ορθή επεξεργασία κάθε πληροφορίας. Με αυτόν τον τρόπο, η Αθήνα λειτούργησε ως το σημείο αναφοράς για την καθιέρωση μιας στιβαρής και ενιαίας μεθοδολογίας ανάκτησης δεδομένων, η οποία εφαρμόστηκε επιτυχώς σε όλες τις επόμενες πόλεις.

Στιγμιότυπο Εκτέλεσης 1 Αθήνα

```
NAME : Athens
COUNTRY : Greece
POPULATION : 643,452
AREA : 38.964 km²
REGION : Attica
COORDINATES : 37°59'03"N 23°43'41"E / 37.98417°N 23.72806°E
ELEVATION : 338 m
TIMEZONE : UTC+2
TEMPERATURE(YEAR HIGH/YEAR LOW/YEAR AVG) : 44.8(112.6)/-6.5(20.3)/19.2(66.6)
```

2. Θεσσαλονίκη

Επιλέχθηκε λόγω της πολυεπίπεδης πληροφορίας που περιέχει το Infobox της καθώς και λόγο της πληροφορίας Region η οποία αποτελείται από παραπάνω από μια λεξείς. Ο κώδικας σχεδιάστηκε έτσι ώστε να απομονώνει με ακρίβεια τον πληθυσμό, αγνοώντας ενδιάμεσες παραπομπές (όπως βιβλιογραφικούς δείκτες σε αγκύλες) ή δευτερεύοντα στατιστικά στοιχεία (όπως η πληθυσμιακή πυκνότητα). Με αυτόν τον τρόπο, επιτεύχθηκε ο σαφής διαχωρισμός των δεδομένων, ακόμα και όταν αυτά εμφανίζονται σε πολύ κοντινή απόσταση στο κείμενο.

Στιγμιότυπο Εκτέλεσης 2 Θεσσαλονίκη

```
NAME : Thessaloniki
COUNTRY : Greece
POPULATION : 319,045
AREA : 19.307 km²
REGION : Central Macedonia
COORDINATES : 40°38’25”N 22°56’08”E / 40.6403°N 22.9356°E
ELEVATION : 250 m
TIMEZONE : UTC+2
TEMPERATURE(YEAR HIGH/YEAR LOW/YEAR AVG) : 43.3(109.9)/-12.6(9.3)/16.8(62.3)
```

3. Νέα Ιωνία

Η Νέα Ιωνία επιλέχθηκε ώστε να γίνει γενίκευση του κώδικα για ονόματα πόλεων που αποτελούνται από περισσότερες από μία λέξεις. Παράλληλα, αναπτύχθηκε η λογική αναγνώρισης πολλαπλών ετικετών (όπως η επανάληψη του όρου "Municipality" για την έκταση και τον πληθυσμό), επιτρέποντας στο πρόγραμμα να πλοηγείται με ακρίβεια στα σωστά πεδία πληροφοριών βάσει των συμφραζομένων της σελίδας.

Στιγμιότυπο Εκτέλεσης 3 Νέα Ιωνία

```
NAME : Nea Ionia
COUNTRY : Greece
POPULATION : 64,611
AREA : 4.421 km²
REGION : Attica
COORDINATES : 38°2’N 23°45’E / 38.033°N 23.750°E
ELEVATION : 130 m
TIMEZONE : UTC+2
Some information could not be found on the page.
```

4.2. Φάση Ελέγχου

Σε αυτή τη φάση ελέγξαμε τον κώδικα που αναπτύξαμε προηγουμένως σε ένα ευρύτερο δείγμα πόλεων οι οποίες δεν είχαν ληφθεί υπόψη κατά τον αρχικό σχεδιασμό των κανονικών εκφράσεων, με στόχο την αξιολόγηση της ακρίβειας και της γενίκευσης του προγράμματος.

1. **Λάρισα (με πίνακα θερμοκρασίας)**

Στιγμιότυπο Εκτέλεσης 4 Λάρισα

```
NAME : Larissa
COUNTRY : Greece
POPULATION : 164,095
AREA : 335.98 km²
REGION : Thessaly
COORDINATES : 39°38.5’N 22°25’E / 39.6417°N 22.417°E
ELEVATION : 70 m
TIMEZONE : UTC+2
TEMPERATURE(YEAR HIGH/YEAR LOW/YEAR AVG) : 45.5(113.9)/-21.6(-6.9)/15.9(60.6)
```

2. Τρίκαλα (χωρίς πίνακα θερμοκρασίας)

Στιγμιότυπο Εκτέλεσης 5 Τρίκαλα

```
NAME : Trikala
COUNTRY : Greece
POPULATION : 78,605
AREA : 607.59 km²
REGION : Thessaly
COORDINATES : 39°33’N 21°46’E / 39.550°N 21.767°E
ELEVATION : 115 m
TIMEZONE : UTC+2
Some information could not be found on the page.
```

3. Καρδίτσα (χωρίς πίνακα θερμοκρασίας)

Στιγμιότυπο Εκτέλεσης 6 Καρδίτσα

```
NAME : Karditsa
COUNTRY : Greece
POPULATION : 55,979
AREA : 647.4 km²
REGION : Thessaly
COORDINATES : 39°22’N 21°55’E / 39.367°N 21.917°E
ELEVATION : 108 m
TIMEZONE : UTC+2
Some information could not be found on the page.
```

4. Πάτρα (με πίνακα θερμοκρασίας)

Στιγμιότυπο Εκτέλεσης 7 Πάτρα

```
NAME : Patras
COUNTRY : Greece
POPULATION : 215,922
AREA : 334.9 km²
REGION : Western Greece
COORDINATES : 38°15’N 21°44’E / 38.250°N 21.733°E
ELEVATION : 10 m
TIMEZONE : UTC+2
TEMPERATURE(YEAR HIGH/YEAR LOW/YEAR AVG) : 40.8(105.4)/-0.9(30.4)/19.3(66.8)
```

5. Ιωάννινα (χωρίς πίνακα θερμοκρασίας)

Στιγμιότυπο Εκτέλεσης 8 Ιωάννινα

```
NAME : Ioannina
COUNTRY : Greece
POPULATION : 113,978
AREA : 403.32 km²
REGION : Epirus
COORDINATES : 39°39’49”N 20°51’08”E / 39.66361°N 20.85222°E
ELEVATION : 480 m
TIMEZONE : UTC+2
Some information could not be found on the page.
```

6. Χανιά (χωρίς πίνακα θερμοκρασίας)

Στιγμιότυπο Εκτέλεσης 9 Χανιά

```
NAME : Chania
COUNTRY : Greece
POPULATION : 111,375
AREA : 351.3 km²
REGION : Crete
COORDINATES : 35°31’N 24°1’E / 35.517°N 24.017°E
ELEVATION : 20 m
TIMEZONE : UTC+2
Some information could not be found on the page.
```

4.3. Φάση Αξιολόγησης

Από τις παραπάνω δοκιμές παρατηρήσαμε πως το πρόγραμμά μας κατάφερε να εξάγει με επιτυχία και ακρίβεια το 100% των βασικών δεδομένων (Όνομα, Χώρα, Πληθυσμός, Έκταση, Περιοχή,

Συντεταγμένες, Υψόμετρο, Χρονική Ζώνη) από την κάθε ιστοσελίδα καθώς εμφάνισε αντίστοιχο μήνυμα για τις ιστοσελίδες που δεν περιείχαν πλήρες πίνακα θερμοκρασίας.